Lesson 13 Teacher's Checklist / Prep Guide

Complete before class

VIDEO GAME CREATION

This Week: A Hit Gets a Reaction

Since Lesson 4, an enemy hit by a bullet has vanished instantly - functional, but not satisfying. This week adds two visual touches: a brief hit flash on the enemy right before it's removed, and an independent explosion effect that appears at that spot and plays itself out. This is the first Visual Art / animation-focused lesson of the course.

This lesson genuinely restructures each engine's hit-handling code, more than most weeks - read the Handout for your demo engine (Godot) closely before class, and skim the others so you can recognize each engine's shape of the same idea when circulating.

- ☐ Read the Teacher's Script's timing-mechanism comparison before class - three different engine families solve 'pause before finishing the kill' three different ways this week, and it's worth understanding all three before fielding student questions.

Self-Contained Lesson Assets

Two new placeholder assets are provided in this lesson's Assets folder: explosion_placeholder.png (a simple burst/star shape) and enemy_invader_white.png (the same silhouette as Lesson 4's enemy_invader_red.png, recolored solid white - this is the flash sprite for Godot, Unity, GameMaker, and Unreal, swapped in briefly on hit instead of tinted, for the same reason this course's sprite-swap Recolor from Lesson 2 exists: a multiply-based color tint can only darken/mix with a sprite's existing color, so tinting an already-colored enemy toward white produces no visible change at all - swapping to a genuinely white sprite is the only way to get an actual white flash). Unreal's own BP_Enemy Material hits this exact trap too, since Lesson 4 built its Color parameter as a multiply against the enemy's real texture (the same technique Godot's modulate and Unity's SpriteRenderer.color use) - Set Vector Parameter Value("Color", White) would be just as invisible there as it is in Godot or Unity, which is why Unreal's own flash swaps in M_Enemy_White (built from this sprite) with a Set Material node instead. TIC-80 students draw their own white flash sprite directly in the Sprite Editor, and Scratch uses its own color-cycling Graphic Effect (not a multiply tint, so it isn't affected either); GDevelop's Level 1 uses the Flash extension's blink action, which just toggles visibility and works regardless of sprite color. 001 Game Creator's built-in Actor - Flash action is a genuine engine feature, not a manual tint, so it's unaffected too.

- ☐ Confirm explosion_placeholder.png and enemy_invader_white.png are both present and import cleanly on the demo laptop before class.

- ☐ Godot / Unity / GameMaker specifically: confirm the enemy visibly turns white (not just brighter or unchanged) for the flash - if it doesn't, the flash sprite likely wasn't imported/assigned, and the code silently fell back to a no-op tint.

Continuity From Lesson 12

- ☐ Remind students to open their own [YourName]Game project from Lesson 12, not create a new one.
- ☐ If a student's Lesson 12 project is lost or broken, have a pre-built "Lesson 12 complete" starter available per engine (from the Lesson 12 Teacher's Keys) so they aren't blocked.

Engine Verification

- ☐ Confirm the demo laptop's Lesson 11-12 exercises (multiple waves, all four sounds) still work for all 8 engines.
- ☐ Test a full round on every engine before class: a hit should flash the enemy briefly, an explosion should appear and clean itself up, and score/lives/wave/win-lose logic should all still behave exactly as before - none of today's changes should alter WHEN score or the win-check actually resolve, only how the enemy's removal looks.
- ☐ GameMaker and TIC-80 specifically: confirm the enemy's actual destruction now happens slightly after the hit is registered (via an Alarm or countdown), not in the same instant - this is intentional this week, not a regression.
- ☐ 001 Game Creator (Windows demo laptop): confirm the Actor – Flash action is available in your installed version and produces a visible flash; if the enemy is destroyed before the flash completes, that's a known, acceptable limitation this week (see the Handout's troubleshooting note) - the explosion effect is this engine's primary required visual.

Materials

- ☐ Print or queue digital copies of all 8 Student Handouts (or confirm students can access their engine's version electronically).
- ☐ Have the 8 Teacher's Keys on hand (instructor reference only - do not distribute) in case a student needs a worked example beyond Level 1.
- ☐ Bring: demo laptop, projector cable/adapter, handout copies.

Known Gotchas This Week

- The most likely mistake: delaying or moving the score/win-check logic itself, rather than only delaying the enemy's final visual removal - the two must stay separate. Watch for this specifically since it's an easy line to blur.
- A close second: forgetting to reset a flashed sprite/texture back to normal (relevant mainly for any engine where an object could theoretically be reused, e.g. an object pool) - low risk this week since every enemy that flashes is destroyed a fraction of a second later, but worth a mention if a student asks.
- This is a genuinely fun week for students - encourage them to actually watch their explosions rather than just confirming they exist, and to compare notes with classmates on other engines during the share-out.
- If a cluster finishes Level 1 quickly, point them at Level 3's animation-timing ideas rather than rushing ahead to Lesson 14's material.